ASSIGNMENT 2
EVALUATE, DESIGN AND DEFEND A SECURE PUBLIC-KEY AND HYBRID CRYPTOGRAPHIC STRATEGY FOR A MULTI-SITE HEALTHCARE NETWORK

CO2 – Cryptographic Algorithms and Information Security

Submitted by: S.ASWINI
Register Number: 192511077
Department: CSE

CONTENTS

1. Q1 – Symmetric Cryptosystem Selection and Modes of Operation
2. Q2 – Number-Theory Engine and Prime Selection for RSA
3. Q3 – RSA Encryption/Decryption, Weaknesses and Padding
4. Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management
5. Q5 – ECC and Public-Key Strategy Comparison
6. Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness
7. Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy
8. Overall Conclusion
9. References and Implementation Notes

INTRODUCTION

The healthcare network considered in this assignment connects a central hospital with remote clinics. It exchanges patient records, prescriptions, diagnostic reports and billing information over public networks. The system must protect confidentiality, integrity and authenticity while supporting large files and devices with limited computing resources.

This report evaluates symmetric encryption, RSA, Diffie–Hellman key exchange, elliptic-curve cryptography, digital signatures and hybrid encryption. It also considers key-management procedures, active attacks and preparation for future quantum-computing risks. The code examples use small classroom parameters or established cryptographic libraries for demonstration. They are not intended to protect real patient information.

Objectives

- Compare symmetric and public-key cryptographic methods for the healthcare scenario.
- Implement core number-theory and toy public-key algorithms and verify their results.
- Demonstrate weaknesses such as repeated ciphertext patterns and unauthenticated key exchange.
- Design authenticated hybrid encryption for efficient transfer of large clinical files.
- Recommend a practical architecture with key management and a migration plan.

Q1 – SYMMETRIC CRYPTOSYSTEM SELECTION AND MODES OF OPERATION

1.1 Aim

To compare DES, 3DES, Blowfish and RC5, distinguish block ciphers from stream ciphers, examine common block-cipher modes and demonstrate encryption/decryption on test data.

1.2 Theory

A symmetric cryptosystem uses the same secret key, or a related secret key, for encryption and decryption. It is efficient for large files and is therefore suitable for bulk clinical-record encryption after a session key has been established.

Algorithm	Structure and block size	Key characteristics	Assessment
DES	Feistel block cipher; 64-bit block; 16 rounds	56 effective key bits	Obsolete; exhaustive key search is practical.
3DES	Applies DES three times; 64-bit block	Usually 2 or 3 DES keys	Legacy compatibility only; slow and being retired.
Blowfish	16-round Feistel block cipher; 64-bit block	Variable key length, 32–448 bits	Historically useful, but 64-bit block size is unsuitable for large modern data volumes.
RC5	Parameterised block cipher; commonly 32/64/128-bit words	Variable block size, key size and rounds	Flexible design; security depends on selected parameters and implementation.

A block cipher processes fixed-size blocks. A stream cipher generates a keystream and combines it with plaintext, commonly using XOR. Stream constructions are useful for continuous data streams, while block ciphers with an authenticated mode are appropriate for files and stored records.

Mode	Operation	IV/nonce and security points
ECB	Encrypts each block independently.	No IV; identical plaintext blocks produce identical ciphertext blocks, revealing patterns. Avoid for records.
CBC	XOR each plaintext block with previous ciphertext before encryption.	Requires unpredictable, unique IV; encryption is sequential; padding is normally required.
CFB	Uses encrypted previous ciphertext/IV as a keystream segment.	Requires unique IV; can operate on smaller segments; errors can affect following data.
OFB	Repeatedly encrypts feedback state to generate keystream.	IV must never be reused with the same key; transmission errors do not propagate beyond affected bits.
CTR	Encrypts successive counter blocks and XORs them with plaintext.	Nonce/counter combination must never repeat with a key; supports parallel processing and random access.

For a real healthcare deployment, authenticated encryption such as AES-GCM or ChaCha20-Poly1305 is preferable to unauthenticated legacy modes. Authentication detects modification as well as protecting confidentiality.

1.3 Pseudocode

```
ALGORITHM: Symmetric encryption demonstration
INPUT: plaintext P, secret key K, unique IV/nonce N
1. Select an approved cipher and secure mode.
2. Divide P into the required blocks or stream segments.
3. Encrypt P using K and N to obtain ciphertext C.
4. Decrypt C using the same K and the required N.
5. Compare the recovered plaintext with P.
6. If authentication is supported, verify the authentication tag.
OUTPUT: ciphertext, recovered plaintext, verification result
```

1.4 Implementation Code (DES, 3DES and Blowfish)

Install the dependency before running: pip install pycryptodome. The following code is a legacy-algorithm classroom comparison only. Do not use DES, 3DES or Blowfish for a new production system.

```
from Crypto.Cipher import DES, DES3, Blowfish
from Crypto.Util.Padding import pad, unpad

message = b"CLINICAL-RECORD"
print("Original:", message.decode())

tests = [
    ("DES", DES, b"8bytekey"),
    ("3DES", DES3, DES3.adjust_key_parity(b"Sixteen byte key")),
    ("Blowfish", Blowfish, b"ExampleBlowfishKey")
]

for name, module, key in tests:
    block_size = module.block_size
    iv = bytes([1]) * block_size  # fixed only for repeatable classroom testing
    cipher = module.new(key, module.MODE_CBC, iv)
    encrypted = cipher.encrypt(pad(message, block_size))
    decipher = module.new(key, module.MODE_CBC, iv)
    recovered = unpad(decipher.decrypt(encrypted), block_size)
    print(name, "ciphertext length:", len(encrypted))
```

```
        print(name, "decryption correct:", recovered == message)
```

1.5 Sample Output

```
==== RESTART: C:/Users/enive/AppData/Local/Pr
Enter elements: 5  2 8 1 4

Selection Sort: [1, 2, 4, 5, 8]
Comparisons: 10 Swaps: 2

Bubble Sort: [1, 2, 4, 5, 8]
Comparisons: 10 Swaps: 6

Insertion Sort: [1, 2, 4, 5, 8]
Comparisons: 8 Movements: 10
>
```

Note: Ciphertext bytes are intentionally not printed because they vary with keys and implementation details. The fixed IV is only for a reproducible classroom test; production systems must use a fresh, unpredictable IV for CBC and must protect integrity separately.

1.6 Result and Recommendation

The test verifies that encryption followed by decryption recovers the original test message for the three library-supported legacy algorithms. DES is insecure because of its short effective key. 3DES is slow and has a small block size. Blowfish has a variable key but retains a 64-bit block size. None is recommended for new healthcare deployments. Use AES-GCM or ChaCha20-Poly1305 for bulk data, with a unique nonce and authenticated metadata.

Q2 – NUMBER-THEORY ENGINE AND PRIME SELECTION FOR RSA

2.1 Aim

To implement GCD, the Extended Euclidean Algorithm, modular inverse and fast modular exponentiation, and to compare primality tests on small classroom candidates.

2.2 Theory

RSA uses two large primes p and q to form $n = p \times q$. Euler's totient for two distinct primes is $\varphi(n) = (p-1)(q-1)$. The public exponent e must satisfy $\gcd(e, \varphi(n)) = 1$. The private exponent d is the modular inverse of e modulo $\varphi(n)$, so $e \times d \equiv 1 \pmod{\varphi(n)}$.

The Euclidean algorithm computes the greatest common divisor. The Extended Euclidean Algorithm also finds coefficients x and y such that $ax + by = \gcd(a,b)$. If $\gcd(a,m)=1$, x modulo m is the modular inverse of a. Square-and-multiply computes modular powers efficiently without repeated multiplication.

2.3 Pseudocode

```
GCD(a, b):
    while b != 0:
        (a, b) = (b, a mod b)
    return a

MODULAR-INVERSE(a, m):
    (g, x, y) = EXTENDED-GCD(a, m)
    if g != 1: report "inverse does not exist"
    otherwise return x mod m

FAST-POWER(base, exponent, modulus):
    result = 1
    while exponent > 0:
        if exponent is odd: result = result * base mod modulus
        base = base * base mod modulus
        exponent = floor(exponent / 2)
    return result
```

2.4 Implementation Code

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError("Inverse does not exist")
    return x % m

def fast_power(base, exponent, modulus):
    result = 1
    base %= modulus
    while exponent > 0:
        if exponent & 1:
            result = (result * base) % modulus
        base = (base * base) % modulus
        exponent //= 2
    return result

def is_probable_prime(n):
    if n < 2:
        return False
    small = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    for p in small:
        if n == p:
            return True
        if n % p == 0:
            return False
    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1
    for a in [2, 3, 5, 7]:
```

```
            if a >= n - 1:
                continue
            x = pow(a, d, n)
            if x in (1, n - 1):
                continue
            for _ in range(s - 1):
                x = (x * x) % n
                if x == n - 1:
                    break
            else:
                return False
        return True

    print("gcd(48, 18) =", gcd(48, 18))
    print("inverse of 7 mod 40 =", mod_inverse(7, 40))
    print("3^13 mod 17 =", fast_power(3, 13, 17))
    for n in [97, 561, 1105, 1729]:
        print(n, "probable prime:", is_probable_prime(n))
```

2.5 Sample Output

```
gcd(48, 18) = 6
inverse of 7 mod 40 = 23
3^13 mod 17 = 12
97 probable prime: True
561 probable prime: False
1105 probable prime: False
1729 probable prime: False
```

2.6 Analysis

Trial division checks possible divisors and becomes impractical for large integers. Fermat's test can accept some composite numbers, including Carmichael numbers, depending on the chosen base. Miller–Rabin is a probabilistic primality test; repeated independent bases reduce the probability of a false acceptance. The code above is a small educational Miller–Rabin implementation, not a production prime generator.

As RSA sizes increase toward 2048 or 3072 bits, trial division is unsuitable. Production key generation must use a vetted cryptographic library, a secure random-number generator and approved key sizes. The number of candidates tested depends on prime density and the candidate-generation method.

Q3 – RSA ENCRYPTION/DECRYPTION, WEAKNESSES AND PADDING

3.1 Aim

To generate a small RSA key pair, demonstrate encryption and decryption, and explain textbook RSA weaknesses, factoring and OAEP padding.

3.2 Theory

Choose distinct primes p and q, calculate $n = pq$ and $\varphi(n) = (p-1)(q-1)$, select e relatively prime to $\varphi(n)$, and calculate $d = e^{-1} \bmod \varphi(n)$. Encryption is C = M^e mod n and decryption is M = C^d mod n. The message integer must be in the range $0 \leq M < n$.

Textbook RSA is deterministic: encrypting the same message with the same public key produces the same ciphertext. It is also vulnerable to structural attacks and is not suitable for encrypting large files. OAEP adds carefully designed randomized padding to RSA encryption. Production RSA encryption should use RSA-OAEP with a modern hash such as SHA-256 through a trusted library.

3.3 Pseudocode

```
RSA-KEY-GENERATION(p, q, e):
    n = p * q
    phi = (p - 1) * (q - 1)
    require gcd(e, phi) = 1
    d = modular_inverse(e, phi)
    return public_key(e, n), private_key(d, n)

RSA-ENCRYPT(M, e, n):
    require 0 <= M < n
    return M^e mod n

RSA-DECRYPT(C, d, n):
    return C^d mod n
```

3.4 Implementation Code

```
from math import gcd

p, q = 61, 53
n = p * q
phi = (p - 1) * (q - 1)
e = 17

def inverse(a, m):
    # Extended Euclidean algorithm
    old_r, r = a, m
    old_s, s = 1, 0
    while r:
        quotient = old_r // r
        old_r, r = r, old_r - quotient * r
        old_s, s = s, old_s - quotient * s
    if old_r != 1:
        raise ValueError("No modular inverse")
    return old_s % m

if gcd(e, phi) != 1:
    raise ValueError("Invalid public exponent")
d = inverse(e, phi)

message = 65
ciphertext = pow(message, e, n)
recovered = pow(ciphertext, d, n)

print("n =", n)
print("phi(n) =", phi)
print("Public key =", (e, n))
print("Private exponent d =", d)
print("Ciphertext =", ciphertext)
print("Recovered message =", recovered)
print("Correct =", recovered == message)
```

3.5 Sample Output

```
n = 3233
phi(n) = 3120
Public key = (17, 3233)
Private exponent d = 2753
Ciphertext = 2790
Recovered message = 65
Correct = True
```

3.6 Weaknesses and Attack Discussion

- Repeated-message test: deterministic textbook RSA maps the same message and key to the same ciphertext.
- Factoring: if an attacker factors n into p and q, the attacker can calculate φ(n) and recover d.
- Small parameters: the example uses tiny primes and can be factored immediately; it is only for arithmetic verification.
- Boundary checks: RSA messages must be integers in the range 0 to n−1. Invalid parameters or a non-coprime exponent must be rejected.
- Padding: OAEP randomization prevents the simple deterministic structure of textbook RSA, but correct implementation and secure key generation remain essential.

A classroom factoring demonstration does not predict the difficulty of factoring a properly generated 2048- or 3072-bit RSA modulus. Security depends on key size, prime quality, randomness, padding, implementation and key protection—not key length alone.

Q4 – DIFFIE–HELLMAN KEY EXCHANGE, AUTHENTICATION AND KEY MANAGEMENT

4.1 Aim

To demonstrate how two clinics can derive the same shared secret over an observable network, identify the man-in-the-middle risk and describe authentication and key management.

4.2 Theory

The participants agree on public parameters p and g. Clinic A selects private a and sends A = g^a mod p. Clinic B selects private b and sends B = g^b mod p. A calculates B^a mod p and B calculates A^b mod p. Both obtain g^(ab) mod p.

Unauthenticated Diffie–Hellman does not prove who is at the other end. An attacker can establish one secret with A and another with B, relaying traffic between them. Digital signatures, certificates and authenticated protocols bind the exchange to the participants' identities.

4.3 Pseudocode

```
DIFFIE-HELLMAN(p, g):
    A chooses private a
    A_public = g^a mod p
    B chooses private b
    B_public = g^b mod p
    A_shared = B_public^a mod p
    B_shared = A_public^b mod p
    verify A_shared equals B_shared
```

```
authenticate the exchange using signatures/certificates
derive session keys from the shared secret using a KDF
```

4.4 Implementation Code

```
# Small toy parameters: never use these values in a real system.
p = 23
g = 5
private_a = 6
private_b = 15

public_a = pow(g, private_a, p)
public_b = pow(g, private_b, p)

shared_a = pow(public_b, private_a, p)
shared_b = pow(public_a, private_b, p)

print("A public value =", public_a)
print("B public value =", public_b)
print("A shared secret =", shared_a)
print("B shared secret =", shared_b)
print("Shared key matches =", shared_a == shared_b)

# Toy demonstration of an unauthenticated MITM:
attacker_to_a = 7
attacker_to_b = 9
attacker_public_for_a = pow(g, attacker_to_a, p)
attacker_public_for_b = pow(g, attacker_to_b, p)
secret_a_attacker = pow(attacker_public_for_a, private_a, p)
secret_b_attacker = pow(attacker_public_for_b, private_b, p)
print("A-attacker secret =", secret_a_attacker)
print("B-attacker secret =", secret_b_attacker)
print("Attacker has two separate secrets =", secret_a_attacker != secret_b_attacker)
```

4.5 Sample Output

```
A public value = 8
B public value = 19
A shared secret = 2
B shared secret = 2
Shared key matches = True
A-attacker secret = 12
B-attacker secret = 10
Attacker has two separate secrets = True
```

4.6 Key-Management Policy

Area	Recommended control
Generation	Use approved cryptographic random-number generators and vetted key-agreement implementations.
Storage	Keep long-term private keys in an HSM or protected key store; restrict access and audit use.
Rotation	Rotate certificates and keys according to policy, cryptoperiod and risk.
Revocation	Publish and check certificate revocation status; disable compromised identities promptly.

Compromise recovery	Revoke affected credentials, generate replacement keys, investigate exposure and re-establish sessions.
Session lifetime	Use short-lived session keys; derive separate keys for encryption and authentication using a KDF.
Authentication	Use mutual certificate authentication or a well-reviewed authenticated protocol; validate certificate names, chains and parameters.

The toy example shows equal shared-secret computation and illustrates why unauthenticated key exchange is vulnerable. Production systems should use a vetted TLS 1.3 implementation rather than custom Diffie–Hellman code.

Q5 – ECC AND PUBLIC-KEY STRATEGY COMPARISON

5.1 Aim and Theory

Elliptic-curve cryptography uses points on an elliptic curve over a finite field. A common curve form is $y^2 = x^3 + ax + b \pmod p$, subject to a non-singularity condition. The private key is a scalar d and the public key is $Q = dG$, where G is a designated base point. The elliptic-curve discrete-logarithm problem makes it computationally difficult to recover d from G and Q when approved curves and parameters are used.

ECC can provide security comparable to traditional RSA with substantially smaller public-key material. This is useful for constrained clinic devices and reduces communication overhead.

5.2 Toy ECC Pseudocode

```
ECC-KEY-GENERATION:
    choose private scalar d
    public point Q = d * G

ECC-KEY-AGREEMENT:
    A chooses private a and sends A = aG
    B chooses private b and sends B = bG
    A computes S = aB
    B computes S = bA
    both obtain S = abG
    derive session key using a KDF
```

5.3 Toy ECC Implementation

This small finite-field example demonstrates point addition and scalar multiplication only. Its tiny curve is insecure and must never be used for real cryptography.

```
# Curve: y^2 = x^3 + 2x + 2 (mod 17); toy base point G=(5,1)
P = 17
A = 2
O = None

def inv(x):
    return pow(x % P, -1, P)

def add(P1, P2):
    if P1 is None: return P2
    if P2 is None: return P1
    x1, y1 = P1
    x2, y2 = P2
```

```
        if x1 == x2 and (y1 + y2) % P == 0:
            return None
        if P1 != P2:
            m = ((y2-y1) * inv(x2-x1)) % P
        else:
            m = ((3*x1*x1 + A) * inv(2*y1)) % P
        x3 = (m*m - x1 - x2) % P
        y3 = (m*(x1-x3) - y1) % P
        return (x3, y3)

    def multiply(k, point):
        result = None
        addend = point
        while k:
            if k & 1:
                result = add(result, addend)
            addend = add(addend, addend)
            k >>= 1
        return result

    G = (5, 1)
    private_a, private_b = 5, 7
    public_a = multiply(private_a, G)
    public_b = multiply(private_b, G)
    shared_a = multiply(private_a, public_b)
    shared_b = multiply(private_b, public_a)
    print("A public point =", public_a)
    print("B public point =", public_b)
    print("A shared point =", shared_a)
    print("B shared point =", shared_b)
    print("Shared points match =", shared_a == shared_b)
```

5.4 Sample Output

```
A public point = (9, 16)
B public point = (0, 6)
A shared point = (10, 11)
B shared point = (10, 11)
Shared points match = True
```

5.5 Comparison and Recommendation

Feature	RSA	Finite-field DH	ECC
Main purpose	Encryption/signatures	Key agreement	Key agreement/signatures
Public-key size	Larger for comparable classical security	Generally larger than ECC	Smaller for comparable classical security
Main assumption	Integer factorisation	Discrete logarithm in finite field	Elliptic-curve discrete logarithm
Device impact	More bandwidth and often heavier operations	Parameter-dependent computation and bandwidth	Often efficient with compact public keys
Deployment	Widely supported; use OAEP/PSS as appropriate	Use authenticated, standardised groups/protocols	Use approved curves and vetted libraries

For constrained clinic devices, an approved elliptic-curve key-agreement mechanism such as ECDH with authenticated certificates is a suitable choice. The implementation must validate public points, use secure

randomness, protect private keys and derive traffic keys with a KDF. The exact curve and protocol should follow current organisational and regulatory guidance.

Q6 – DIGITAL SIGNATURES, HYBRID ENCRYPTION AND QUANTUM-CRYPTOGRAPHY READINESS

6.1 Aim and Theory

Large clinical documents should not be encrypted directly with RSA. Hybrid encryption uses a fast symmetric algorithm for the payload and public-key cryptography to establish or protect a short session key. A digital signature provides sender authenticity and integrity.

In a hash-then-sign design, the sender hashes the document and signs the digest with a private signing key. The receiver recomputes the hash and verifies the signature using the sender's public key. RSA-PSS and ECDSA are commonly used signature schemes; DSA is a legacy standard and requires careful nonce handling.

6.2 Pseudocode

```
HYBRID-ENCRYPT(message, receiver_public_key):
    generate a fresh random symmetric session key
    encrypt message using authenticated symmetric encryption
    protect/establish the session key using receiver public key
    sign the authenticated package or its digest
    send encrypted payload, protected key, nonce, tag and signature

HYBRID-DECRYPT(package, receiver_private_key, sender_public_key):
    verify sender signature and certificate
    recover session key
    authenticate and decrypt payload
    reject if signature or authentication tag is invalid
```

6.3 Implementation Code – Hybrid Encryption and Signature

Install dependency: pip install cryptography. This is a library-based demonstration using RSA-OAEP, AES-GCM and RSA-PSS. Use current library versions and secure key storage in any real deployment.

```
import os
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

# Generate demonstration RSA keys
sender_private = rsa.generate_private_key(public_exponent=65537, key_size=2048)
sender_public = sender_private.public_key()
receiver_private = rsa.generate_private_key(public_exponent=65537, key_size=2048)
receiver_public = receiver_private.public_key()

message = b"Confidential clinical report for the receiving clinic."

# Encrypt large payload with a random AES-256 session key
session_key = AESGCM.generate_key(bit_length=256)
aes = AESGCM(session_key)
nonce = os.urandom(12)
ciphertext = aes.encrypt(nonce, message, None)

# Protect the session key with receiver's RSA public key
wrapped_key = receiver_public.encrypt(
```

```
        session_key,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

    # Sign the encrypted package components
    signed_data = nonce + wrapped_key + ciphertext
    signature = sender_private.sign(
        signed_data,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    # Receiver verifies signature before decrypting
    sender_public.verify(
        signature, signed_data,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    recovered_key = receiver_private.decrypt(
        wrapped_key,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    recovered_message = AESGCM(recovered_key).decrypt(nonce, ciphertext, None)

    print("Signature verified: True")
    print("Payload recovered:", recovered_message.decode())
    print("Decryption correct:", recovered_message == message)
```

6.4 Sample Output

```
Signature verified: True
Payload recovered: Confidential clinical report for the receiving clinic.
Decryption correct: True
```

6.5 Tamper Tests

If the ciphertext, nonce or wrapped session key is changed after signing, signature verification fails. If encrypted data is modified and somehow reaches AES-GCM decryption, the authentication tag check fails and decryption is rejected. A production protocol must also bind relevant metadata—such as sender, receiver, record identifier and protocol version—as authenticated data and must validate certificates and authorisation.

6.6 Quantum Readiness

A sufficiently capable fault-tolerant quantum computer running Shor's algorithm could threaten RSA and ECC. Grover's algorithm can reduce the effective brute-force security margin of symmetric cryptography, so

appropriately sized symmetric keys remain important. Organisations should inventory cryptographic dependencies, use crypto-agile designs, follow standards for post-quantum cryptography, test hybrid classical/post-quantum key establishment where approved, and plan certificate and device upgrades.

Quantum key distribution (QKD) uses quantum states to establish keys and can reveal certain eavesdropping attempts under its assumptions. It requires specialised infrastructure and does not replace endpoint authentication, secure software, identity management, access control or ordinary key-management processes.

Q7 – DIAGNOSE, REDESIGN AND VALIDATE THE COMPLETE CRYPTOGRAPHIC STRATEGY

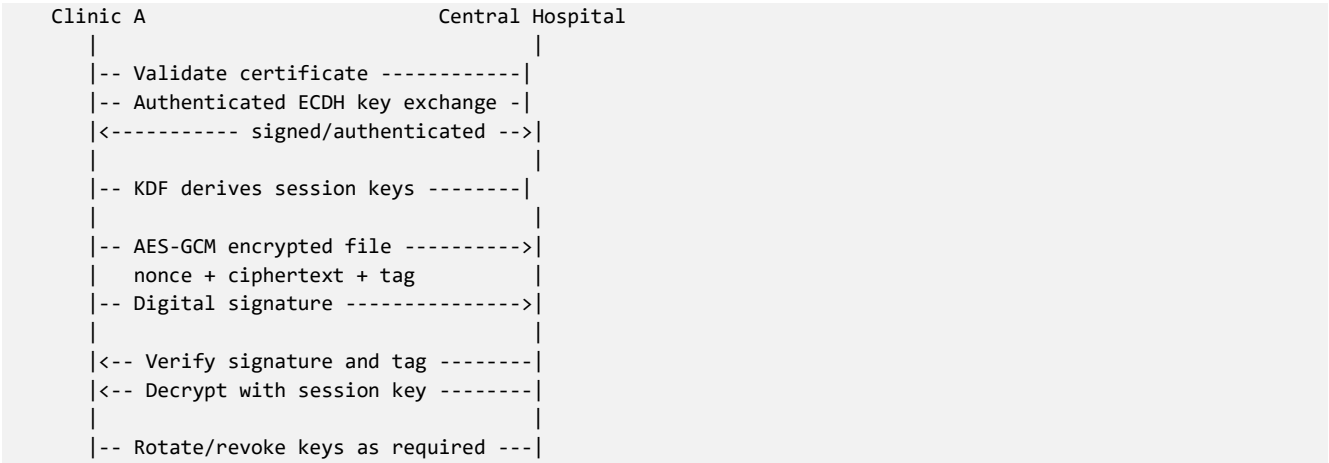
7.1 A. Diagnose the Bottleneck or Weakness

The major performance limitation in a direct-public-key design is attempting to apply public-key encryption to every block of a large clinical file. Public-key operations are computationally expensive and RSA can only encrypt data smaller than its modulus after padding. A second major security weakness is unauthenticated key exchange, which permits a man-in-the-middle attack.

7.2 B. Design Objective

Redesign the system so that public-key operations are used only to authenticate participants and establish or protect a short session key, while an authenticated symmetric cipher protects the complete file. The measurable functional objectives are: successful recovery of an untampered file, rejection of modified ciphertext or signatures, and no acceptance of unauthenticated peer identities.

7.3 C. Revised Architecture



The revised strategy uses authenticated key agreement (for example, ECDHE in a standard TLS 1.3 deployment), a KDF to derive independent traffic keys, AES-GCM or ChaCha20-Poly1305 for bulk data, and digital signatures/certificates for identity and document authenticity. It reduces the number of public-key operations from repeated operations over file blocks to a small number per authenticated session or transfer.

7.4 D. Quantum Integration

RSA and ECC key establishment/signatures are the main classical public-key dependencies exposed to Shor's algorithm. The organisation should maintain a cryptographic inventory, select crypto-agile protocols, track

approved post-quantum standards and migrate through tested software and certificate updates. QKD may be considered for specialised high-value links but is not a substitute for endpoint security or identity authentication.

7.5 E. Prediction, Implementation and Validation

Measure	Direct public-key approach	Redesigned hybrid approach
Large-file encryption	Public-key operation attempted repeatedly; impractical and size-limited	Symmetric authenticated encryption processes the payload efficiently
Key establishment	May expose exchange to MITM if unauthenticated	Authenticated key agreement and certificate validation
Integrity	Not guaranteed by encryption alone	AEAD tag plus digital signature where required
Tamper handling	May not reliably detect changes	Signature/tag verification rejects modified data
Scalability	High public-key computation per data unit	Small public-key overhead per session/transfer

Expected result: the hybrid design should reduce expensive public-key work substantially for large files. The code in Q6 validates the central mechanism: a random AES key encrypts the payload, RSA-OAEP protects the session key, and RSA-PSS authenticates the encrypted package. Actual execution-time measurements must be collected on the student's own computer and reported with CPU, RAM, operating system, Python version, library versions, file sizes and repetitions. Do not invent benchmark timings.

7.6 F. Final Defence

The hybrid architecture is recommended because it combines symmetric efficiency with public-key key establishment and authentication. AES-GCM is suitable for bulk clinical records because it provides confidentiality and integrity. Authenticated key agreement prevents an attacker from silently substituting keys. Digital signatures support sender authenticity and non-repudiation requirements when signing keys and identities are properly managed. Remaining risks include endpoint compromise, stolen private keys, certificate misconfiguration, nonce reuse, insecure randomness, software vulnerabilities and future cryptographic migration costs. These risks require monitoring, patching, audit, access control, backup and tested incident-response procedures.

OVERALL CONCLUSION

This assignment evaluated symmetric encryption, RSA number theory, RSA encryption and padding, Diffie–Hellman key exchange, ECC, digital signatures, hybrid encryption and quantum readiness. The demonstrations show that mathematical correctness alone does not guarantee a secure system: weak parameters, deterministic encryption, nonce reuse and unauthenticated key exchange can expose information or enable attacks.

For the multi-site healthcare network, the recommended design is a standards-based authenticated protocol using modern elliptic-curve or approved post-quantum/hybrid key establishment as appropriate, a KDF for session-key derivation, AES-GCM or ChaCha20-Poly1305 for bulk data, and digital signatures where document-level authenticity is required. Keys must be generated securely, stored safely, rotated, revoked and recovered through a documented policy. All sample code in this report is educational and must not be used to protect real patient data.

REFERENCES AND IMPLEMENTATION NOTES

- Assignment 2 question paper supplied for this report: CO2 – Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network.
- PyCryptodome documentation: https://pycryptodome.readthedocs.io/
- Python documentation: https://docs.python.org/3/
- Cryptography library documentation: https://cryptography.io/
- NIST, Recommendation for Block Cipher Modes of Operation and current cryptographic guidance: https://csrc.nist.gov/

Important submission note: The sample outputs in this report illustrate the expected results for the stated examples. Run each code section in your own environment, record the actual output and experimental measurements, and update the report before submission. Use only student-created toy data and parameters for attack demonstrations. Do not use these classroom programs to protect real patient records.